

Capitolo 35 — PROT-006 — Branch Ownership & Baseline Sync

Pubblico	Lettori tecnici e non tecnici del WCM Process & Memory Book
Versione	FROZEN-35
Data master	2026-08-30
Stato	FROZEN
Source path	wcm/documentation/process-memory-book/ chapters/35_prot_006_branch_ownership_baseline_sync.md
Source SHA	20142c8
Release	2026-08-31T20:48:50.990Z

GitHub main è la source of truth. Questo PDF è una release derivata: non costituisce approvazione né autorità.

PARTE VII — Il Libro dei Protocolli WCM Stato: FROZEN **Data:** 2026-08-30 **Scope:** WCM generale, domain-agnostic

35.0 Isolare il lavoro senza creare due verità

Quando più attività evolvono nello stesso sistema, può essere utile separare temporaneamente un lavoro dal resto.

La separazione protegge ciò che è stabile mentre qualcosa viene provato, revisionato o modificato. Ma la stessa separazione, se dura troppo o perde il collegamento con la baseline comune, crea un problema più grave di quello che voleva risolvere: **due versioni dello stesso sistema iniziano a vivere come se fossero entrambe la realtà corrente.**

PROT-006 — Branch Ownership & Baseline Sync governa precisamente questo rischio.

La sua idea centrale è:

Il lavoro può essere isolato temporaneamente; la source of truth operativa non deve diventare temporanea anch'essa.

Nel modello corrente WCM, **main** è il trunk operativo condiviso. Una branch non rappresenta normalmente un progetto autonomo e non deve trasformarsi in una seconda baseline permanente. Esiste quando serve isolamento tecnico temporaneo e deve avere fin dall'inizio una ragione e una condizione di uscita.

Per un lettore non tecnico, una branch può essere immaginata come una **copia di lavoro controllata**: si prende una base comune, si prova o si prepara qualcosa separatamente, poi si decide che cosa deve tornare nella base comune e che cosa invece deve essere chiuso o conservato soltanto come evidenza.

Il punto non è evitare le copie di lavoro. È evitare che una copia di lavoro diventi, per inerzia, una realtà concorrente.

35.1 Il problema che PROT-006 risolve

Supponiamo che un'organizzazione possieda un insieme di regole e materiali correnti.

Una persona deve sperimentare una modifica importante. Per prudenza crea un ambiente separato. L'esperimento dura alcuni giorni; nel frattempo la baseline comune continua a evolvere.

A quel punto esistono due storie:

- la baseline comune contiene cambiamenti nuovi;
- l'ambiente isolato contiene il lavoro sperimentale.

Se le due storie vengono lasciate divergere troppo a lungo, quando arriva il momento di ricongiungerle non basta più chiedersi quale file sia più recente. Bisogna capire **quale significato debba sopravvivere**.

Il rischio cresce ulteriormente se l'ambiente isolato rimane aperto dopo che la sua funzione è terminata. Potrebbe diventare una fonte consultata da qualcuno come se fosse ancora autorevole. Potrebbe contenere regole generali che non sono mai state promosse. Oppure potrebbe essere considerato “il posto del progetto”, mentre il resto del WCM evolve altrove.

PROT-006 previene questa deriva con tre idee complementari:

1. **ownership logica chiara** — sapere dove deve vivere stabilmente ogni tipo di contenuto;
2. **isolamento temporaneo motivato** — creare una branch solo quando esiste una ragione concreta;
3. **sincronizzazione e uscita esplicite** — impedire che la branch sviluppi una baseline autonoma.

35.2 Branch: prima il concetto, poi il termine tecnico

Nel linguaggio di Git, una *branch* è una linea di lavoro separata che parte da uno stato conosciuto e può evolvere indipendentemente per un certo periodo.

Ma per comprendere PROT-006 non serve conoscere Git.

È sufficiente distinguere due concetti:

BASELINE COMUNE

= ciò che rappresenta lo stato operativo corrente

COPIA DI LAVORO ISOLATA

= spazio temporaneo in cui una modifica può evolvere senza alterare subito la baseline

WCM usa **main** come baseline operativa condivisa. La branch è lo spazio isolato, quando serve.

Il protocollo supera quindi il modello “un progetto = una branch permanente”. La separazione principale dei progetti avviene logicamente nella struttura dei contenuti, non attraverso linee di sviluppo permanenti e indipendenti.

In termini semplici: **il confine tra due aree di lavoro non richiede necessariamente due realtà tecniche separate.**

35.3 Ownership: sapere dove appartiene qualcosa

La parola *ownership* non indica qui il possesso personale di un file.

Indica il suo **luogo logico di appartenenza**.

Nella baseline corrente:

- `wcm/**` contiene metodo, governance, architettura e componenti generali WCM;
- `projects/<project>/**` contiene gli elementi specifici del singolo progetto.

Questa distinzione serve a rispondere a una domanda fondamentale:

Questo cambiamento appartiene al metodo generale o a un contesto specifico?

Se la risposta non è chiara, anche una sincronizzazione tecnicamente perfetta può produrre un risultato semanticamente sbagliato.

L'ownership viene quindi prima del semplice spostamento di contenuti.

35.4 Quando si attiva il protocollo

PROT-006 entra in gioco quando il lavoro coinvolge una branch esistente o quando si sta valutando se crearne una.

Il trigger tipico può essere rappresentato così:

```

SERVE MODIFICARE / SPERIMENTARE / REVISIONARE
      ↓
SERVE DAVVERO ISOLAMENTO TEMPORANEO?
      ↓
      PROT-006

```

Il protocollo canonico indica condizioni concrete nelle quali una branch può essere giustificata, tra cui:

- un POC o esperimento che può fallire o sporcare lo stato operativo;
- un refactoring o una modifica strutturale non ancora accettata;
- un lavoro che richiede review prima di entrare nel trunk;
- una modifica temporanea a un componente condiviso;
- necessità tecniche legate a CI/CD, release, security o compliance;
- un'altra motivazione esplicita e documentata.

La nascita di un nuovo progetto, da sola, **non è una ragione sufficiente**.

35.5 Gli input

Per decidere correttamente non basta sapere che “serve una branch”.

Gli input utili sono almeno concettualmente:

- la baseline corrente su **main**;
- lo scope del lavoro da isolare;
- la ragione dell'isolamento;
- l'ownership logica dei contenuti coinvolti;
- l'eventuale dipendenza da componenti generali WCM;
- la condizione che dirà quando l'isolamento può terminare;
- lo stato corrente della branch, se esiste già.

Questi input consentono di evitare una branch senza destinazione.

Una copia di lavoro senza una exit condition non è davvero temporanea: è soltanto una seconda realtà che non ha ancora dichiarato di esserlo.

35.6 Primo gate: serve davvero una branch?

Il primo decision point è il più semplice e spesso il più importante.

```
LAVORO DA ESEGUIRE
  ↓
SERVE ISOLAMENTO TECNICO TEMPORANEO?
├ NO → resta sul trunk nel confine logico corretto
└ SÌ → branch temporanea con motivo + exit condition
```

Questo gate impedisce la proliferazione automatica di branch.

Separare tecnicamente ogni attività può sembrare ordinato, ma nel tempo aumenta il costo di sincronizzazione e rende più difficile capire quale stato sia davvero corrente.

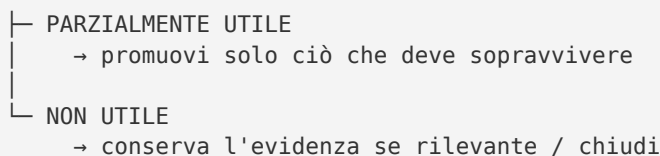
PROT-006 preferisce quindi il trunk condiviso come condizione normale e l'isolamento come eccezione motivata.

35.7 L'exit condition: sapere come finisce prima di iniziare

Ogni branch temporanea deve sapere fin dall'inizio come termina.

Il protocollo canonico distingue tre esiti generali:

```
BRANCH TEMPORANEA
  ↓
RISULTATO / EVIDENZA
  ↓
CLASSIFICAZIONE
├ ACCETTATO
│   → merge o promozione selettiva su main
└
```



La parola importante è **selettiva**.

Una branch può contenere contemporaneamente:

- evidenza dell'esperimento;
- implementazioni temporanee;
- parti che hanno valore generale;
- parti che non devono sopravvivere.

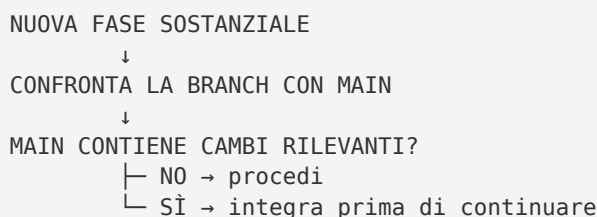
Per questo chiudere una branch non significa necessariamente riversare tutto nel trunk.

Significa classificare il risultato e consolidare soltanto ciò che appartiene alla baseline futura.

35.8 Il Temporary Branch Sync Gate

Una branch temporanea continua a vivere mentre **main** evolve.

Prima di una nuova fase sostanziale, PROT-006 richiede quindi un controllo di sincronizzazione:



Questo è il **Temporary Branch Sync Gate**.

Il suo scopo non è mantenere due ambienti identici in ogni istante. Se fossero sempre identici, l'isolamento non avrebbe senso.

Lo scopo è impedire che il lavoro isolato continui per troppo tempo assumendo una baseline ormai superata.

Un esempio pedagogico può aiutare.

Immaginiamo che una persona prepari una revisione separata di un regolamento. Nel frattempo il regolamento ufficiale cambia in due punti importanti. Prima di proseguire con una nuova sezione della revisione, è ragionevole confrontare la copia con l'ultima versione ufficiale. Altrimenti potrebbe costruire ulteriore lavoro sopra una premessa che non esiste più.

L'esempio non introduce una nuova regola WCM: rende semplicemente intuitivo il motivo del gate.

35.9 Le due direzioni della sincronizzazione

La sincronizzazione non è un gesto unico. Esistono due direzioni con scopi diversi.

Dalla branch verso **main**

Quando un risultato deve sopravvivere, la domanda non è “come copio tutto?”, ma:

| Che cosa di questo lavoro appartiene alla baseline operativa?

Se la branch contiene insieme evidenza sperimentale e cambi generalizzabili, il protocollo esclude il merge indiscriminato come comportamento predefinito. Occorre prima acceptance e, quando il cambiamento riguarda il metodo, il processo appropriato di promozione della baseline.

Da **main** verso la branch

Quando il lavoro isolato dipende da una baseline più recente, **main** viene integrato nella branch.

Questo mantiene chiara la direzione di autorità: la branch può assorbire la baseline corrente per continuare a lavorare, ma non diventa per questo una baseline autonoma.

35.10 Experimental override: una regola generale provata in isolamento

Un caso delicato si presenta quando un esperimento richiede di modificare temporaneamente un elemento generale WCM.

La modifica può essere utile per il POC e tuttavia non essere ancora parte della baseline.

PROT-006 mantiene la distinzione:

```
MODIFICA GENERALE IN BRANCH
    ↓
    SPERIMENTALE
    ↓
    RISULTATO / EVIDENZA
    ↓
    ACCETTAZIONE E PROCESSO DI PROMOZIONE APPROPRIATO
    ↓
    SE GENERALIZZABILE → MAIN
    ALTRIMENTI → NON DIVENTA BASELINE
```

Questo passaggio è essenziale perché impedisce che una regola sperimentale resti indefinitamente in un luogo laterale e venga poi trattata, magari mesi dopo, come se fosse già canonica.

Quando l'isolamento termina, anche la doppia source of truth deve terminare.

35.11 I conflitti non sono soltanto differenze di testo

Quando due linee di lavoro modificano la stessa area, può comparire un conflitto.

Una lettura puramente tecnica potrebbe ridurre il problema a: “quale versione scegliamo?”.

PROT-006 richiede prima una **classificazione semantica**:

- **TRUNK_ONLY** — prevale lo stato operativo corrente di **main**;
- **BRANCH_ONLY** — il cambiamento isolato viene preservato se è ancora necessario;
- **SHARED_EVOLVED_BOTH** — entrambi i lati contengono evoluzione valida e serve una fusione semantica esplicita;
- **AMBIGUOUS** — il conflitto non viene risolto automaticamente: si ferma e viene escalato all'autorità prevista.

Il motivo è semplice.

Due testi diversi possono entrambi contenere conoscenza valida. Scegliere automaticamente “il nostro” o “il loro” lato può cancellare una parte della realtà senza che nessuno se ne accorga.

Il conflitto è quindi prima un problema di significato e solo dopo un problema di meccanica Git.

35.12 Output: che cosa deve essere vero alla fine

L'output di PROT-006 non è semplicemente “branch sincronizzata”.

La disciplina è rispettata quando lo stato complessivo torna leggibile:

- **main** rappresenta lo stato operativo corrente;
- ogni contenuto ha un confine logico coerente;
- ogni branch aperta ha uno scopo temporaneo documentabile;
- la branch non è diventata una seconda source of truth permanente;
- gli eventuali cambi generalizzabili sono consolidati tramite il processo appropriato;
- al termine dell'isolamento, la branch può essere chiusa o archiviata secondo l'esito.

L'output reale è quindi **una sola baseline operativa comprensibile, anche dopo un periodo di lavoro isolato**.

35.13 Failure mode

PROT-006 esiste perché alcune failure sono facili da produrre e difficili da vedere subito.

Branch permanente per abitudine

Una branch nasce per un lavoro specifico ma resta aperta indefinitamente. Con il tempo acquisisce una propria storia e comincia a essere trattata come source of truth parallela.

Branch creata soltanto perché nasce un progetto

L'isolamento tecnico viene usato come sostituto della separazione logica. Il risultato è una moltiplicazione di baseline da mantenere allineate.

Nessuna exit condition

La branch ha un inizio ma non una definizione di fine. Nessuno sa quando il suo contenuto debba essere promosso, scartato o archiviato.

Sync troppo tardivo

Il lavoro isolato continua a lungo mentre `main` evolve. Quando si prova a ricongiungere le due linee, il problema non è più un semplice aggiornamento ma una ricostruzione semantica.

Merge indiscriminato

Tutto ciò che esiste nella branch viene trasferito nella baseline, compresi artefatti sperimentali che non avevano superato acceptance o promozione.

Risoluzione automatica di un conflitto semantico

Si sceglie meccanicamente un lato anche quando entrambi contengono conoscenza valida. La sincronizzazione riesce tecnicamente ma perde significato.

Override sperimentale lasciato vivere fuori dal trunk

Una modifica generale dimostrata utile non viene consolidata nel luogo autorevole; oppure, al contrario, viene trattata come regola generale senza passare dal processo appropriato.

35.14 Relazioni con gli altri elementi WCM

PROT-006 non lavora da solo.

Il protocollo canonico richiama esplicitamente:

- **DEC-005 — Trunk-Based Project Operating Model**, che definisce il modello operativo di riferimento nel quale `main` è il trunk condiviso;
- **PROC-004 — Evidence → Baseline Promotion**, da applicare quando un'evidenza sperimentale relativa al metodo deve essere valutata per una possibile promozione;
- **PROT-001 — Git & Working Tree Safety**, che protegge il lavoro locale e impone prudenza prima di operazioni Git potenzialmente distruttive;

- **PROT-003 — Direct Before Delegate**, che governa la scelta della capacità più diretta e adeguata quando occorre eseguire il lavoro.

La relazione più importante è concettuale:

PROT-006 governa dove e per quanto tempo il lavoro può divergere; gli altri elementi governano come operare in sicurezza, come valutare ciò che emerge e come promuovere ciò che deve diventare baseline.

35.15 Maturity e limiti

La baseline canonica corrente di PROT-006 è marcata:

| VALIDATED BY GOVERNANCE / EVOLVED 2026-08-11

Questa qualifica significa che il protocollo appartiene alla governance corrente ed è stato evoluto rispetto a una regola precedente ormai superata.

Non significa che ogni possibile topologia di repository, ogni organizzazione o ogni contesto operativo esterno sia stato validato sul campo.

Il protocollo stesso mantiene infatti un limite esplicito: se esigenze future di security o compliance richiederanno una separazione più forte, la topologia dovrà essere rivalutata tramite una decisione esplicita.

La regola corrente è quindi una baseline operativa governata, non una pretesa di universalità.

35.16 Source map

Per questo capitolo il Technical Truth Pass è stato ancorato alla fonte canonica:

- `wcm/process-book/protocols/PROT-006_BRANCH_OWNERSHIP_BASELINE_SYNC.md`

Il protocollo canonico richiama inoltre, come relazioni necessarie alla sua interpretazione:

- DEC-005 — Trunk-Based Project Operating Model;
- PROC-004 — Evidence → Baseline Promotion;
- PROT-001 — Git & Working Tree Safety;
- PROT-003 — Direct Before Delegate.

Il capitolo non modifica questi elementi e non introduce nuove regole WCM. Traduce editorialmente la baseline corrente di PROT-006 in una forma comprensibile anche a un lettore non tecnico.

35.17 La regola da ricordare

Se di questo capitolo restasse una sola idea, sarebbe questa:

| Isola il lavoro quando serve, non la verità. Una branch è temporanea; la baseline operativa deve tornare una sola.

L'isolamento è utile proprio perché può finire.

Quando non finisce, smette di essere protezione e diventa divergenza.